

Panoptes: A P2P Zero-Trust Cryptographic Engine for Efficient Mental Poker

Antonio López Vivar
April 2026

Abstract

Traditional digital card games rely on centralized servers, introducing catastrophic single points of failure, while decentralized Web3 alternatives fail to achieve real-time viability due to prohibitive block latency. This paper introduces Panoptes, a highly optimized, hybrid Zero-Trust cryptographic engine that enforces microsecond-latency decentralized consensus for the CoronaPoker peer-to-peer network. Assuming a strict Ring-0 adversary model, Panoptes treats the host operating system and the Java Virtual Machine (JVM) as fundamentally compromised. We detail a bifurcated architecture utilizing a hardened native airgap that leverages OS-level stealth allocators to process ciphertexts without leaving plaintext residue in the managed heap. To mitigate Direct Memory Access (DMA) attacks and forensic analysis, Panoptes implements a multiplexed decoy memory topology (The Vault), secured by hardware-level page guarding and offline TOCTOU decryption. The protocol entirely replaces traditional commutative encryption with the deterministic Hand Commitment Megapacket, a flat-buffer payload leveraging X25519 KEM, Additive Secret Sharing, and ChaCha20-Poly1305 to ensure Byzantine fault tolerance without majority voting. We present formal implementations of our micro-architectural defenses, including Mixed Boolean-Arithmetic (MBA) for constant-time execution, direct cross-platform syscalls bypassing `libc`, OS-level DACL lockdowns, PEB cloaking, and asynchronous SipHash-2-4 binary attestation. Furthermore, we introduce a multithreaded Deadman Switch to detect CPU cycle drift via `RDTSC`. Evaluated under an exhaustive 40-point “Total Siege” adversarial framework, the engine demonstrates unparalleled resilience against hardware breakpoints, kernel introspection, inline hooking, and temporal drift attacks.

1 Introduction

1.1 The Trust Deficit in Centralized Architectures

In the landscape of competitive digital card games, the absolute confidentiality and integrity of the game state are the primary prerequisites for economic viability. Historically, online poker platforms have utilized a centralized client-server topology. In this model, the server acts as an omniscient authority, responsible for entropy generation, deck permutation, and the selective routing of private information (hole cards) to clients via TLS-encrypted channels.

This centralized paradigm is structurally fragile. It forces the user to grant unconditional trust to the service provider, its physical infrastructure, and its internal personnel. Real-world incidents, such as the Absolute Poker and Ultimate Bet scandals, provided empirical proof of this failure: insiders with elevated database access utilized “superuser” accounts to view opponents’ cards in real-time, leading to the fraudulent extraction of millions of dollars [2]. Furthermore, the reliance on a single server-side Pseudo-Random Number Generator (PRNG) creates a catastrophic single point of failure. A statistically biased or poorly seeded PRNG allows sophisticated adversaries to reconstruct the deck permutation through passive observation of public cards—a process known as “PRNG grinding”—which bypasses all transport-layer security measures.

1.2 The Web3 Paradox: Latency vs. Secrecy

To resolve the centralized trust problem, recent industry efforts have looked toward Web3 and Smart Contracts. However, distributed ledgers introduce a fundamental conflict between public transparency and game secrecy. In a public blockchain (e.g., Ethereum), the state is fully transparent, meaning hole cards cannot be stored on-chain without encryption. If cards are encrypted on-chain, the commutative keys must be managed off-chain, re-introducing trust issues.

Furthermore, the performance bottlenecks of decentralized Virtual Machines (EVM) are insurmountable for real-time play [4]. With block confirmation times ranging from 12 seconds to several minutes, and the prohibitive cost of “gas” for every state transition, blockchain-native poker is economically and technically non-viable for high-frequency competitive gameplay.

1.3 The Panoptes Vision: Native Zero-Trust

Panoptes is designed to bridge the gap between the speed of centralized systems and the trustlessness of decentralized protocols. It is not a global blockchain; it is a localized P2P consensus engine that treats the client’s own hardware as a hostile battleground. By shifting the focus from “protecting the client environment” to “hardening the cryptographic engine within a compromised client,” Panoptes ensures that if a node is tampered with, it mathematically loses the ability to produce valid signatures for the mesh.

The engine relies on a dual-pillar strategy:

1. **Part I: The Zero-Trust Protocol:** A mathematical framework utilizing X25519 KEM and the Hand Commitment Megapacket to replace the server’s authority.
2. **Part II: The Micro-Architectural Shield:** A systems-engineering framework that uses MBA-obfuscation [23], PEB-hashing, and cache-aligned memory slots to deny the adversary the ability to read or modify the state even with Ring-0 access.

2 State of the Art and Related Work

The challenge of achieving fairness and secrecy in decentralized card games has been a focal point of cryptographic research for decades. Previous approaches suffer from severe computational overhead, reliance on trusted hardware, or prohibitive network latency.

2.1 Mental Poker and Commutative Encryption

The foundational SRA Mental Poker protocol, introduced by Shamir, Rivest, and Adleman in 1981 [1], provided the first theoretical proof that a fair card game could be played over a network without a trusted third party. The protocol relies on commutative encryption schemes, allowing cards to be encrypted and decrypted in any order. Subsequent improvements by Barnett and Smart [3] introduced verifiable shuffling using homomorphic encryption. However, these protocols require $O(N^2)$ computationally expensive asymmetric exponentiation operations for deck generation and card drawing. In a high-stakes P2P mesh, this overhead renders real-time, fluid gameplay impossible. Panoptes entirely replaces commutative encryption with an X25519 KEM and symmetric ChaCha20 cipher, reducing state transition latency to the sub-millisecond range.

2.2 Trusted Execution Environments (TEEs)

Industry alternatives often attempt to solve the trust deficit by anchoring game logic within Trusted Execution Environments (TEEs), such as Intel SGX or ARM TrustZone. In these models,

a hardware-locked enclave executes the dealing logic and attestations. However, the assumption of hardware invulnerability has been repeatedly shattered by micro-architectural side-channel attacks, including Foreshadow [16], Plundervolt, and SGXaxe, which allow Ring-0 adversaries to extract encryption keys directly from the enclave. Panoptes assumes the underlying silicon is inherently vulnerable, securing the state entirely through software obfuscation, dynamic key assembly, and strict memory volatility, thus eliminating the dependency on proprietary hardware trust anchors.

2.3 Blockchain and zk-SNARKs

Modern Web3 gaming architectures utilize Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs) to prove valid state transitions without revealing hidden information (e.g., in games like *Dark Forest* [17]). While mathematically sound, the computational cost of generating a single zero-knowledge proof requires seconds to minutes of CPU time. When combined with the block latency of decentralized ledgers [4], synchronous, high-frequency betting rounds become unplayable. Panoptes abstracts the global ledger into an ephemeral, high-frequency micro-ledger (`HAND_STATE_BLOCKCHAIN`) confined to the lifespan of a single hand, operating natively at CPU speeds via Symmetric-Key Swarm Signatures.

3 System and Adversary Model

3.1 High-Level Architecture Overview

To resolve the fundamental conflict between a hostile operating system and the need for microsecond-latency cryptography, Panoptes bifurcates its architecture into an untrusted transport layer and a hardened cryptographic enclave. This architecture operates under the assumption that the host environment (the Java Virtual Machine and the OS kernel) is actively attempting to extract the cryptographic state.

The system is structurally divided into four interconnected domains:

- **The Untrusted Frontend (Java/JVM):** The application layer manages the graphical user interface, network sockets, and OS-level I/O. In the Panoptes threat model, the JVM is treated strictly as a compromised transport medium, vulnerable to Garbage Collection (GC) memory orphanage and JDWP introspection [24].
- **The JNI Airgap:** The strict boundary between the managed runtime and the native engine. Panoptes enforces a “Copy-Transform-Annihilate” protocol via the Java Native Interface (JNI). Ciphertexts are physically copied from the JVM heap into stealth-allocated native buffers, neutralizing the JVM’s ability to track cryptographic payloads or pin memory arrays.
- **The Native Enclave (C):** A freestanding, statically opaque execution environment devoid of standard `libc` dependencies. Within this enclave, all game state mutations, X25519 Key Encapsulation, and Sponge KDF operations occur. The enclave protects its physical memory utilizing a multiplexed decoy topology (The Vault) and evades OS telemetry via direct system calls and Mixed Boolean-Arithmetic (MBA) obfuscation.
- **The P2P Mesh Layer:** The decentralized network topology where peers exchange stateless, flat-buffer data structures. The mesh achieves consensus through the exchange of encrypted Megapackets, N-MAC Swarm Signatures, and Cryptographic Receipts, entirely eliminating the need for a centralized dealer or a global blockchain ledger.

By isolating the core game logic within the Native Enclave, Panoptes guarantees that the vulnerable Frontend only ever handles opaque ciphertexts and cryptographic hashes, effectively blinding a Ring-0 adversary to the underlying plaintext game state.

3.2 The Ring-0 Adversary Model

Current anti-cheat technologies typically operate by attempting to elevate the security software to the kernel level (Ring-0) to monitor the execution environment. However, this “escalation war” is fundamentally flawed in a Zero-Trust context. Deploying defensive kernel drivers introduces severe user privacy concerns, system instability, and merely shifts the attack surface without solving the underlying trust deficit.

Crucially, Panoptes does not attempt to fight the adversary for control of the OS kernel. The engine operates entirely within user space (Ring-3). Its architectural triumph lies in achieving Ring-0 resilience from a Ring-3 boundary; it utilizes mathematics, volatile memory barriers, and spatial obfuscation to isolate and obliterate the cryptographic state before kernel-level introspection tools can asynchronously harvest the memory.

Panoptes assumes a **worst-case adversary model** where the attacker is not a casual user, but a malicious actor with administrative (`root`/`SYSTEM`) privileges and physical access to the hardware. Under this model, the adversary is capable of executing the following high-tier attacks:

- **Asynchronous Memory Carving (DMA):** Utilizing custom PCIe-based Direct Memory Access hardware (e.g., PCILeech) to directly map the system’s physical RAM [21]. This method completely bypasses the OS kernel and its virtual memory protections, allowing the attacker to extract the plaintext game state without triggering software-level hooks.
- **Micro-architectural Side-channels:** Exploiting CPU cache hierarchies (L1 through L3) to measure timing variances [14]. Techniques such as *Flush+Reload* allow an attacker to determine if a specific memory address containing a cryptographic key was accessed.
- **Hardware-Level Instrumentation:** Manipulating the CPU’s hardware debug registers (DR0 through DR3) to set hardware breakpoints. Unlike software breakpoints, these are invisible to standard debug-detection APIs and can pause the execution of the engine.
- **Managed Runtime Introspection:** Exploiting the Java Virtual Machine’s (JVM) inherent debug capabilities [24]. Attackers utilize Java Agents (JVMTI) or JDWP debuggers to decompile bytecode at runtime, monitor the managed heap, and manipulate the game’s logic.

4 Core Cryptographic Primitives

The Panoptes engine operates as a strictly deterministic, localized state machine. To ensure provably fair play in an environment completely devoid of a central authority, the core primitives must solve three fundamental problems: non-deterministic entropy generation on hostile hardware, the creation of an immutable P2P consensus seed, and the mathematical elimination of statistical bias in deck generation.

4.1 Micro-Architectural Jitter and Hybrid Entropy Extraction

Traditional entropy sources, such as the operating system’s PRNG (`/dev/urandom` or `SystemFunction036`), are highly susceptible to interception or starvation under a Ring-0 adversary model.

If the primary OS entropy source fails or is intercepted, the Panoptes engine falls back to a **Micro-Architectural Jitter Accumulator**. The engine harvests raw entropy by measuring the non-deterministic timing variances inherent in modern CPU Out-of-Order Execution (OoOE)

pipelines [15]. We utilize the `RDTSC` instruction (or `cntvct_el0` on ARM architecture) to sample the high-frequency cycle count during a localized busy-wait loop. Let t_1 and t_2 be the cycle counts before and after the loop. The engine calculates the first-order temporal delta:

$$\Delta t = t_2 - t_1 \quad (1)$$

Rather than relying on a single bit, the engine applies bitwise shifts ($\gg 8, \gg 16$) to the temporal delta Δt and folds the resulting bytes directly into an isolated 32-byte entropy pool (`jitter_pool`). Because these deltas are intrinsically tied to thermal noise, L1/L2 cache misses, and asynchronous scheduler interrupts (provoked via `sched_yield`), they are physically impossible to predict for an external observer.

4.2 Architectural Specification: The 512-bit Sponge Permutation

For all hashing, state accumulation, and Key Derivation Functions (KDF), Panoptes replaces standard rigid hash functions with a custom **Sponge Construction** $\mathcal{Z}$ [9]. This architecture provides flexible input/output boundaries and inherent immunity to length-extension attacks.

The internal state $\mathcal{S}$ is 512 bits wide, structurally partitioned into:

- **Rate** (r): 256 bits, utilized for data absorption and squeezing.
- **Capacity** (c): 256 bits, permanently hidden from the attacker to ensure a formal 2^{128} collision resistance level.

The core permutation function $f : \{0, 1\}^{512} \rightarrow \{0, 1\}^{512}$ utilizes an unkeyed variant of the ChaCha20 round function [7]. The state is initialized with strict constants (e.g., `0x61707865` and `0x3320646e`) treated as a matrix of sixteen 32-bit words, and the transformation consists of 10 double-rounds of the ARX (Add-Rotate-Xor) primitive. During the **Absorption Phase**, input data D is processed in 256-bit blocks:

$$\mathcal{S}_{t+1} = f(\mathcal{S}_t \oplus (D \parallel 0^c)) \quad (2)$$

5 The Hand Commitment Megapacket Protocol

To neutralize the “Last-Actor Paradox” and entirely circumvent the $O(N^2)$ computational overhead of traditional commutative encryption, Panoptes enforces a strict, temporally-locked commit-and-reveal protocol culminating in the **Hand Commitment Megapacket**. The architecture strictly forbids any single peer, including the hand’s dealer (Host), from injecting arbitrary or uncommitted entropy into the game state.

5.1 Phase 1: Zero-Trust Mesh Commitments

Let $P = \{p_1, \dots, p_n\}$ be the set of active peers. Each peer p_i independently generates a 256-bit local secret $s_{local,i}$ via the Micro-Architectural Jitter Accumulator. Using the Sponge KDF, peer p_i derives a public seed $s_{pub,i}$. Peer p_i then broadcasts a cryptographic commitment $C_i = \text{Squeeze}(\mathcal{Z}(s_{pub,i}))$. The protocol execution halts until all n commitments are natively registered in the local `P2P_COMMITMENTS` vault matrix.

By decoupling the internal entropy ($s_{local,i}$) from the public seed ($s_{pub,i}$), peers can safely contribute to the global state without exposing their underlying cryptographic derivation trees.

5.2 Phase 2: Megapacket Assembly and Byte Architecture

In a decentralized topology, the peer acting as the Megapacket generator (the Host) inherently holds asymmetric processing power. Panoptes mathematically neutralizes this by introducing the Master Shuffle Key ($K_{shuffle}$) and Additive Secret Sharing [12]. $K_{shuffle}$ serves a dual cryptographic purpose: it acts as the final irreversible entropy constraint for the deck, and it acts as the symmetric cipher key for the Deck Capsule.

When generating their local entropy ($s_{local,host}$), the Host also derives the top-secret 256-bit $K_{shuffle}$. Because $s_{pub,host}$ and $K_{shuffle}$ are derived from the exact same $s_{local,host}$, the Host is mathematically locked to a specific $K_{shuffle}$ the moment they broadcast their commitment in Phase 1.

The **Hand Commitment Megapacket** is a heavily optimized, flat binary buffer devoid of standard serialization formats (e.g., JSON or Protobuf) to neutralize parser-based memory exploits. Its exact architectural layout is deterministic and natively assembled via pointer arithmetic:

Table 1: Anatomy of the Hand Commitment Megapacket

Segment	Size (Bytes)	Cryptography	Architectural Function
Header	49	Plaintext	Contains <code>HAND_ID</code> (16B), <code>num_players</code> (1B), and the Host’s <code>server_seed</code> (32B) for Phase 1 verification.
Player Routing	$N \times 64$	Plaintext	For each peer: Public Seed (32B) and Target Long-Term PubKey (32B).
KEM Envelopes	$N \times 146$	X25519 + AEAD	Contains the Ephemeral PubKey (32B) and a 114B ChaCha20-Poly1305 ciphertext encapsulating 98 bytes of plaintext: hole cards (2B), <code>HAND_ID</code> (16B), <code>SHUFFLE_KEY_SHARE</code> (32B), and <code>STREET_TOKENS</code> (48B).
Deck Capsule	68	ChaCha20	The 52-card Fisher-Yates permutation and 16B <code>HAND_ID</code> , symmetrically locked by $K_{shuffle}$.
Escrow Ciphertext	5	ChaCha20	The 5 community cards, sequentially locked by the N -of- N threshold Street Keys.
Chaos Signature	16	Poly1305 MAC	Global payload integrity check keyed by the <code>CHAOS_SEED</code> to prevent bit-flipping during transit.

The exact permutation of the deck is governed by the Final Seed (S_{final}), constructed by XORing all verified public seeds with the Host’s secret $K_{shuffle}$:

$$S_{final} = \left(\bigoplus_{i=1}^n s_{pub,i} \right) \oplus K_{shuffle} \quad (3)$$

Immediately after generating the Megapacket, the Host splits $K_{shuffle}$ into n distinct cryptographic shards using XOR additive sharing:

$$K_{shuffle} = \bigoplus_{i=1}^n shard_i \quad (4)$$

Each peer p_i receives their specific $shard_i$ hidden inside their 146-byte AEAD envelope. Crucially, the Host’s plaintext $K_{shuffle}$ and the underlying plaintext deck are instantly obliterated from the physical RAM of the Host utilizing the `secure_wipe` volatile memory barrier. The deck now exists purely as a cryptographically locked capsule distributed across the mesh.

5.3 Mitigation of Modulo Bias: Constant-Time Rejection Sampling

The most statistically sensitive operation in the engine is the conversion of the final entropy pool into the deck permutation via the Fisher-Yates algorithm [11]. Standard digital card games often utilize simple modulo reduction ($R \bmod j$), which introduces severe **Modulo Bias**.

Panoptes completely eliminates this flaw by implementing **Constant-Time Rejection Sampling**. For each card selection, the engine extracts a 16-bit word R from a ChaCha20 keystream. The value R is accepted if and only if it satisfies the strict divisibility threshold, calculated via integer truncation to avoid implementation-defined compiler bugs:

$$R < \lfloor 65536/j \rfloor \times j \quad (5)$$

If R falls into the remainder tail of the distribution, it is discarded, and the engine consumes the next 2 bytes of the keystream. This guarantees that every remaining card possesses an identical probability of $1/j$ of being selected.

5.4 Branchless Integrity Validation

To maintain the integrity of the Zero-Trust execution environment during Megapacket ingestion, the validation of commitments must not leak control-flow information via timing side-channels [18]. Panoptes utilizes Mixed Boolean-Arithmetic (MBA) [23] to compute integrity masks in constant time. Instead of relying on vulnerable branch prediction logic (`if (hash == commitment)`), the engine evaluates the delta:

$$\Delta = \text{Squeeze}(\mathcal{Z}(s_{pub,i})) \oplus C_i \quad (6)$$

$$mask = (\Delta \mid (\sim \Delta + 1)) \gg 31 \quad (7)$$

This branchless *mask* is explicitly tied to the engine’s internal memory state. If any bit of Δ is non-zero, the *mask* evaluates to `0xFFFFFFFF`, irrevocably poisoning the local node’s cryptographic Vault and forcing a silent failure.

6 Key Encapsulation Mechanism (KEM)

The secure distribution of private state (hole cards) in a peer-to-peer topology presents a significant cryptographic bottleneck. Legacy protocols utilizing commutative encryption require $O(N^2)$ exponentiation operations, rendering real-time gameplay impossible. Panoptes circumvents this by utilizing a high-speed Key Encapsulation Mechanism (KEM) based on the X25519 Elliptic Curve Diffie-Hellman (ECDH) function [5].

6.1 X25519 Scalar Multiplication and Subgroup Clamping

Panoptes implements the X25519 key exchange over the Montgomery curve defined by $y^2 = x^3 + 486662x^2 + x$ over the prime field $\mathbb{F}_p$ where $p = 2^{255} - 19$. This specific curve is chosen for its complete immunity to timing side-channels when scalar multiplication is performed via the Montgomery Ladder algorithm [6].

To maintain absolute security against small-subgroup confinement attacks within our Zero-Trust environment, the engine performs manual bit-clamping on all ephemeral private keys. When a 256-bit raw entropy string sk_{raw} is squeezed from the Sponge accumulator to act as an ephemeral scalar, the engine applies the following bitwise constraints directly in the native C stack:

1. The three least significant bits of the first byte are cleared: $sk_{raw}[0] \& = 248$. This ensures the scalar is a multiple of the cofactor (8).
2. The most significant bit of the last byte is cleared: $sk_{raw}[31] \& = 127$.
3. The second most significant bit of the last byte is set: $sk_{raw}[31] | = 64$. This guarantees that the highest set bit is always at a fixed position, preventing timing leaks regarding the scalar’s bit-length.

6.2 Low-Order Point Neutralization (Silent Poisoning)

A critical vulnerability in decentralized ECDH is the injection of low-order public points by a malicious peer. If an adversary transmits a public key PK_{adv} that belongs to a small subgroup of Curve25519, the resulting shared secret K_{ss} will collapse into a predictable set of low-entropy values.

Panoptes implements an active, branchless validation of the scalar multiplication result. The engine evaluates the non-zero status of the shared secret:

$$\Delta_{weak} = \bigvee_{i=0}^{31} K_{ss}[i] \quad (8)$$

If Δ_{weak} evaluates to zero, the engine avoids throwing an exception. Instead, it utilizes an MBA masking function $\mathcal{M}_{poison} = ((\Delta_{weak} | (\sim \Delta_{weak} + 1)) \gg 31) \oplus 1$ to infect the subsequent Key Derivation Function. This ensures that the derived keys are deterministically corrupted, causing the peer to decrypt a randomized “phantom card” that triggers a consensus failure.

6.3 Sponge-Based Key Derivation Function (S-KDF)

The raw shared secret K_{ss} is structurally uniform but not suitable for direct use as a symmetric cipher key. Panoptes treats K_{ss} strictly as an entropy source that must be processed through an “Extract-then-Expand” KDF.

We utilize our 512-bit Sponge permutation $\mathcal{Z}$ to derive the keys. Notably, the engine isolates the derivation by absorbing the shared secret and the ephemeral public key:

$$\mathcal{S}_{512} \leftarrow \text{Squeeze}(\mathcal{Z}(K_{ss} || PK_e)) \quad (9)$$

The 512-bit state $\mathcal{S}$ is partitioned to produce two distinct 256-bit symmetric keys: K_{cipher} for the ChaCha20 keystream, and K_{mac} for the Poly1305 authenticator.

6.4 Strict Temporal Boundaries and Anti-DoS KEM Cache

In a decentralized P2P mesh, nodes are susceptible to Asymmetric Resource Exhaustion attacks. An adversary can flood a target node with thousands of forged KEM payloads, forcing the victim to perform computationally expensive X25519 scalar multiplications.

Panoptes neutralizes this threat through a strict temporal boundary and a pre-computation caching layer:

1. **Temporal Nonce Validation:** Every incoming KEM payload includes a 64-bit micro-architectural timestamp. The engine performs a true branchless verification against `PANOPTES_V_TIMEOUT_SECS` (120 seconds). If a packet arrives outside this strict temporal window, or if its timestamp lies in the future (beyond clock drift tolerance), integer underflow sign-bits are utilized to branchlessly poison the Vault.
2. **Anti-Replay Cache:** Validated nonces are stored in an LRU (Least Recently Used) `g_nonce_cache`. Any duplicate nonce within the 120-second window is instantly rejected, preventing replay attacks.

3. **Anti-DoS Throttle:** The engine implements `dos_cache_check_and_add`, evaluating the sender’s public key. If a peer attempts to saturate the attestation engine, their public key is temporarily blacklisted for 10 seconds (`PANOPTES_DOS_COOLDOWN_SECS`), dropping all subsequent requests in $O(1)$ time before triggering any elliptic curve mathematics.

7 Distributed State and Threshold Consensus

The enforcement of a strictly linear, tamper-evident timeline in a localized P2P mesh requires a state machine capable of resolving asynchronous inputs without a global clock or a Centralized Third Party (TTP). Panoptes achieves this by fragmenting the decryption authority of the game state and anchoring every micro-action into a rapidly evolving, cryptographically entangled hash chain.

7.1 Additive Secret Sharing and Escrow Superposition

To prevent the Host—or any peer—from gaining asymmetric knowledge of the community cards (Flop, Turn, River) prior to the conclusion of the requisite betting rounds, Panoptes utilizes an N -of- N threshold cryptography model based on additive secret sharing [12].

During the genesis of the hand, the Host generates three ephemeral 128-bit Street Keys: K_{flop} , K_{turn} , and K_{river} . The community cards are encrypted via ChaCha20 using these keys, creating the *Escrow Ciphertext* which is packed into the public Megapacket.

Similar to the Master Shuffle Key, the Host cannot retain these Street Keys. Each K_{street} is fragmented into n XOR shards:

$$K_{street} = \bigoplus_{i=1}^n k_{i,street} \quad (10)$$

The Host distributes the corresponding shards ($k_{i,flop}$, $k_{i,turn}$, $k_{i,river}$) into the private KEM envelopes of each peer. Consequently, the community cards exist in a state of cryptographic superposition across the P2P mesh. A street can only be materialized into the L1-cache Vault when the betting round officially closes and every active peer broadcasts their shard to reconstruct the target K_{street} .

7.2 Sequential Token Ratcheting

To categorically prevent out-of-order execution attacks—where a compromised node attempts to broadcast their Turn shard before the Flop is resolved—Panoptes enforces chronological dependency via **Hash-Ratchet Tokenization**.

The effective shard utilized for the decryption of street S is mathematically tethered to the validated plaintext of street $S - 1$. The engine computes the final usable shard $k_{i,S}^{final}$ utilizing the Sponge KDF:

$$k_{i,S}^{final} = \text{Squeeze}(\mathcal{Z}(k_{i,S} \parallel \mathcal{H}(\text{BoardState}_{S-1}))) \quad (11)$$

If an adversary attempts to force the state machine forward prematurely, the lack of a valid BoardState_{S-1} causes the KDF to output a mathematically disjoint key, rendering the premature shard cryptographically inert.

7.3 The Hand State Blockchain (Micro-Ledger)

The chronological integrity of individual player actions is anchored by an internal data structure defined as the `HAND_STATE_BLOCKCHAIN`. Unlike global Web3 ledgers, this is an ephemeral, high-frequency micro-ledger confined to the lifespan of a single hand.

Let A_t be the serialized action payload, ID_t the sender’s public identity, and τ_{micro} the local micro-architectural timestamp. The state H_t evolves via the 512-bit Sponge permutation $\mathcal{Z}$:

$$H_t = \text{Squeeze}(\mathcal{Z}(H_{t-1} \parallel A_t \parallel ID_t \parallel \tau_{micro})) \quad (12)$$

This recursive entanglement guarantees **Forward Immutability**. An adversary cannot replay a "Fold" action from a prior sequence, because the resulting H_t would irreversibly diverge from the consensus mesh.

7.4 N-MAC Swarms and Sliding Window Attestation

To achieve sub-millisecond consensus verification without computationally expensive public-key digital signatures on every action, Panoptes utilizes **N-MAC Swarm Signatures**.

When peer i broadcasts an action block, they append a variable-length payload containing n Poly1305 Message Authentication Codes (MAC). Each MAC in the swarm is keyed uniquely for a specific recipient j using the ephemeral P2P shared secret $K_{p2p,i,j}$:

$$\mathcal{T}_{i,j} = \text{Poly1305}(K_{p2p,i,j}, H_t \parallel A_t) \quad (13)$$

Upon receiving the packet, peer j performs a *Sliding Window Verification*, scanning the swarm for their designated tag $\mathcal{T}_{i,j}$. This provides bilateral cryptographic proof that the sender’s identity is authentic, and both nodes are perfectly synchronized on the exact same H_t blockchain state.

7.5 Zero-Tolerance Consensus and Branchless Poisoning

Most distributed fault-tolerant systems rely on Byzantine algorithms (e.g., PBFT [13]) requiring 2/3 majority voting. In a high-stakes, Zero-Trust game environment, majority voting is vulnerable to Sybil attacks and introduces unnecessary latency. Panoptes eschews voting in favor of **Strict Zero-Tolerance Consensus**.

The native `utilsVerifyHandConsensus` routine evaluates the integrity of the mesh by iterating over all received P2P testaments. For every received testament r_k :

$$err_k = (\mathcal{T}_{local} \oplus \mathcal{T}_{remote}) \vee (H_{t,local} \oplus H_{t,remote}) \quad (14)$$

$$err_{global} \leftarrow err_{global} \vee err_k \quad (15)$$

Following the iteration, the engine generates an MBA mask from the global error state to infect the Vault:

$$v.poison \leftarrow v.poison \vee ((err_{global} \mid (\sim err_{global} + 1)) \gg 31) \quad (16)$$

If even a single bit of a single testament diverges, err_{global} evaluates to a non-zero value, and the mask drives the `v.poison` variable to `0xFFFFFFFF`. This architecture dictates that a hand is only valid if there is absolute, unanimous mathematical agreement.

7.6 The Panoptes Protocol State Machine: Lifecycle of a Hand

The execution flow of a single poker hand within the Panoptes engine is a strictly synchronized, linear state machine.

- **Phase 1: Genesis and Commitment.** All active peers initialize their local cryptographic Vaults. Each peer utilizes the Micro-Architectural Jitter Accumulator to generate a local entropy seed. Peers derive a public seed and broadcast a Sponge-based hash commitment. The protocol stalls until all commitments are securely registered.

- **Phase 2: Encapsulation (The Megapacket).** The Host collects all public seeds and verifies them against the Phase 1 commitments. The Host derives the Master Shuffle Key, computes the Fisher-Yates permutation [11], and extracts the game state. Pocket cards and symmetric key shards (for $K_{shuffle}$ and Street Keys) are encrypted specifically for each peer using the X25519 KEM. The community cards are encrypted into an Escrow Ciphertext. The payload is signed via a custom Chaos MAC (seeded with the Sponge KDF) and broadcast as a single Megapacket.
- **Phase 3: The Action Chain.** As betting commences, peers invoke local actions. Each action is serialized, absorbed into the `HAND_STATE_BLOCKCHAIN`, and broadcast with N-MAC Swarm Signatures. This guarantees every peer is authenticating the action against the exact same chronological timeline.
- **Phase 4: Token Ratcheting.** When a betting round concludes, peers request and broadcast their respective shards for the current street via `stateGetFlopToken` (and subsequent streets). The XOR sum reconstructs the Street Key. Peers use this key to execute a branchless decryption of the Escrow Ciphertext. If a peer attempts to decrypt out-of-order, the mathematical dependency fails, yielding zeroed and poisoned data masks instead of valid cards.
- **Phase 5: Normal Showdown and Forensic Verification.** Upon hand completion, peers natively exchange their `SHUFFLE_KEY_SHARE`. The Panoptes engine autonomously performs a deterministic replay of the entire hand history via `utilsVerifyHandHistory`. It verifies the original Megapacket deck generation and validates the final `HAND_STATE_BLOCKCHAIN` hash. Any divergence mathematically proves an architectural breach, safely terminating the session.

8 Forensic Audit and Cryptographic Receipts

The culmination of a Panoptes hand requires absolute mathematical proof that the game state was not manipulated during its lifecycle. Because the Host holds the asymmetric ability to generate the Megapacket, the protocol demands a rigid verification phase at Showdown known as the Forensic Audit.

8.1 Exception Protocol: The Exit Testament and Vault Lobotomy

In a normal flow, shards and tokens are exchanged organically. However, if a peer abandons the match or crashes, Panoptes executes the `sessionGenerateExitTestament` routine as a failsafe “Will”.

Contrary to traditional protocols that might reveal session keys for verification, Panoptes strictly protects forward secrecy. The testament signs the string literal “EXIT” alongside the peer’s public key using N-MAC Swarm Signatures. It then appends the peer’s `SHUFFLE_KEY_SHARE` and all three `STREET_TOKENS` (Flop, Turn, River) in plaintext to the end of the packet, allowing surviving peers to reconstruct the hand without the dropped node.

Crucially, to prevent post-match physical memory extraction, the engine immediately performs a **Vault Lobotomy**. It physically overwrites the ephemeral X25519 private and public session keys with zeroes (`memset(v.SESSION_PRIV_KEY, 0, 32)`) and explicitly poisons the Vault state (`v.poison = 1`). This guarantees that even if a Ring-0 adversary dumps the RAM after the hand concludes, the KEM decryption keys no longer exist in the physical universe.

8.2 Deterministic State Replay (utilsVerifyHandHistory)

Once the mesh has reconstructed the 256-bit Master Shuffle Key ($K_{shuffle}$) via the XOR-sum of all verified shards, the native engine invokes `utilsVerifyHandHistory`.

The validation sequence operates in $O(1)$ memory without dynamic parsing, directly reading from the original Megapacket:

1. **Seed Validation:** The engine extracts the public seeds embedded in the Megapacket. It verifies that its own locally derived public seed matches the one broadcast by the Host. It then XORs all seeds together, applying a final XOR with the reconstructed $K_{shuffle}$ to derive the true Final Seed ($\mathcal{S}_{final}$).
2. **Symmetry Verification:** The engine performs a fresh Fisher-Yates permutation using $\mathcal{S}_{final}$. It then cross-references this pristine deck against its locally stored Pocket Cards and the dynamically evolved Community Cards (stored in the Vault’s REVEALED_BOARD), factoring in the exact street index masks.
3. **Capsule Decryption:** The engine isolates the encrypted Deck Capsule ($\mathcal{C}_{deck}$) from the Megapacket. It applies the ChaCha20 keystream using $K_{shuffle}$ to decrypt the capsule, validating that the 52-card array originally committed by the Host perfectly matches the newly reconstructed permutation.

If any element—a pocket card, a board card, or a decrypted deck index—fails to align with mathematical perfection, the symmetry validation evaluates to false, branchlessly registering a consensus failure via an accumulated bitwise error mask.

8.3 The Receipt Consensus Matrix

Replaying the deck generation proves the Host did not cheat the shuffle, but it does not prove that the mesh agrees on the final outcome of the betting rounds (e.g., chip balances). To achieve final settlement, Panoptes utilizes Cryptographic Receipts generated via `chainCloseStateAndGetReceipt`.

A Receipt is a compact, rigid binary structure that mathematically binds a peer’s identity to the final chronological state of the hand. Let R_i be the receipt generated by peer p_i . The structure of R_i is defined as:

$$R_i = ID_{pub,i} \parallel \mathcal{H}_{final} \parallel (\mathcal{T}_{i,1} \parallel \cdots \parallel \mathcal{T}_{i,n}) \quad (17)$$

Where:

- $ID_{pub,i}$ is the peer’s 256-bit X25519 public key.
- $\mathcal{H}_{final}$ is the ultimate 256-bit state of the HAND_STATE_BLOCKCHAIN, acting as an irrefutable summary of every action, bet, and street evolution that occurred during the hand.
- $\mathcal{T}_{i,j}$ are N-MAC Swarm Signatures. Peer p_i generates a distinct Poly1305 MAC over $\mathcal{H}_{final}$ for every other peer p_j in the mesh, keyed with their mutual ephemeral P2P shared secret.

8.4 Final Settlement (utilsVerifyHandConsensus)

Upon receiving the N receipts, the native engine executes `utilsVerifyHandConsensus`. This routine sweeps the receipt matrix in a single pass. For each receipt, the engine derives the expected MAC using its own P2P shared secret and searches the swarm via a constant-time sliding window.

It then verifies that the $\mathcal{H}_{final}$ asserted by the peer matches the local Vault’s HAND_STATE_BLOCKCHAIN. By enforcing unanimous agreement on the terminal hash of the micro-ledger, Panoptes guarantees that the resulting chip distribution is cryptographically indisputable, allowing the application layer to safely commit the financial settlement.

9 The Hybrid Zero-Trust Boundary

The theoretical guarantees of the cryptographic protocol established in Part I are rendered obsolete if the physical memory housing the ephemeral keys and state vectors can be trivially inspected by a Ring-0 adversary. Deploying a decentralized poker node within a managed runtime, such as the Java Virtual Machine (JVM), presents a severe architectural paradox. Panoptes resolves this conflict by establishing a physical memory airgap, treating the JVM strictly as a compromised, untrusted transport medium.

9.1 Managed Runtime Physics and Memory Orphanage

The JVM is engineered for developer productivity and memory safety, utilizing a non-deterministic Garbage Collector (GC) to manage the execution heap. However, this architecture is fundamentally antagonistic to constant-time cryptography and Zero-Trust residency control.

When a standard Java application decrypts a network payload into a `byte[]` array, the GC retains absolute authority over the physical memory mapping. During a heap compaction cycle, active objects are relocated to contiguous memory addresses to prevent fragmentation. Critically, the JVM does not zero-out the obsolete memory locations; it merely marks them as unreferenced in the internal allocation tables. We define this vulnerability as **Memory Orphanage**. Under our Ring-0 threat model, an adversary utilizing Direct Memory Access (DMA) hardware [21] can perform asynchronous sweeps of the physical RAM, extracting these plaintext “orphans” long after the cryptographic operation has logically concluded in the application layer.

Furthermore, the JVM exposes deep introspection capabilities via the Java Debug Wire Protocol (JDWP) and Java Native Access (JNA) [24], allowing a compromised host OS to map object references and dynamically inspect variables without triggering execution anomalies.

9.2 The JNI Airgap: Copy-Transform-Annihilate

To neutralize the JVM’s introspection and memory mismanagement, Panoptes implements a rigid “Copy-to-Bunker” protocol via the Java Native Interface (JNI).

Standard performance-optimized JNI wrappers often utilize `GetPrimitiveArrayCritical`, which pins the array in the JVM heap and exposes a direct memory pointer to the native code. While fast, this risks JVM deadlocks and leaves the data persistently visible to Java-level debuggers. Panoptes explicitly rejects this approach, enforcing a physical memory extraction into a hardened C-enclave:

1. **Stealth Allocation:** The native engine provisions an isolated buffer $\mathcal{B}_{native}$ bypassing the standard C library (`malloc/free`), which is heavily monitored by OS-level heap tracking. Panoptes invokes direct system calls to request anonymous memory pages.
2. **Airgapped Extraction:** The ciphertext C is physically copied from the JVM heap into the stealth buffer using the localized routine:

$$C_{native} \leftarrow \text{JNI}::\text{GetByteArrayRegion}(\dots, \mathcal{B}_{native}) \quad (18)$$

3. **Transformation:** All cryptographic mutations occur strictly within the $\mathcal{B}_{native}$ enclave. The physical L1 cache and the CPU registers become the exclusive mediums holding the plaintext.
4. **Annihilation:** Before returning the transformed state to the JVM, the native enclave is subjected to the `secure_wipe` routine, physically obliterating the memory pages.

9.3 Strict JNI Boundary Hardening

Because the JNI boundary acts as the primary attack surface, Panoptes assumes all incoming arrays are actively fuzzed. It enforces strict integer boundary constraints (e.g., rejecting payloads below 49 bytes or above 1 MB) before any cryptographic evaluation occurs. Malformed arrays are instantly rejected, and the Vault is securely wiped, neutralizing buffer-overflow and integer-underflow attacks natively via explicit standard condition evaluations.

9.4 Flat-Buffer Extraction and Anti-Serialization

Modern network protocols typically rely on serialization formats (e.g., JSON, Protobuf) to decode incoming packets. In a Zero-Trust C-enclave, parsing logic is a catastrophic vulnerability. Complex state machines and dynamic heap allocations (`malloc`) required for deserialization introduce massive attack surfaces for memory corruption and heap-spraying exploits.

Aligning with the principles of Language-Theoretic Security (LangSec) [25], Panoptes entirely eliminates deserialization libraries. The engine processes incoming byte arrays using manual pointer arithmetic and fixed-offset extraction (e.g., `packet[16]` for player counts). While the initial JNI boundary requires a single stealth allocation (`STEALTH_HEAP_ALLOC`) to bridge the airgap, the internal extraction operates in $O(1)$ time with zero subsequent allocations. This ensures absolute immunity against object-deserialization vulnerabilities and fundamentally neutralizes padding alignment exploits.

9.5 OS-Level DACL Kernel Lockdown

While manual extraction neutralizes internal parsing vulnerabilities, shielding the physical memory space from external introspection requires adapting to the specific primitives of the host environment. On Windows architectures, Panoptes proactively defends its memory space against external API handles (e.g., `OpenProcess`) utilized by memory scrapers.

Upon initialization, the engine dynamically resolves `SetEntriesInAclA` and `SetSecurityInfo` from `advapi32.dll` via blind FNV-1a hashing. The engine injects a `DENY_ACCESS` Access Control Entry (ACE) targeting the `CURRENT_USER` identity. By explicitly restricting the `PROCESS_VM_READ` and `PROCESS_VM_WRITE` permissions at the OS kernel level, Panoptes mathematically blocks external processes from mapping its memory pages.

9.6 Cryptographic Path-Binding and Offline Forensics Mitigation

To mitigate offline forensic analysis, Panoptes explicitly anchors the encrypted state to its physical location on the host's filesystem. When exporting sensitive session structures (such as `LOCAL_ENTROPY`) to the persistent disk, the engine invokes a `stealth_bind_path_to_key` routine.

This function extracts the absolute path of the executing binary and absorbs it into the `DISK_ENC_KEY` via the Sponge KDF. Consequently, if an adversary steals the serialized session files and attempts to decrypt them in an offline virtual machine, a sandbox, or even a different directory, the Sponge permutation will produce a mathematically disjoint key.

10 Dynamic Obfuscation and OS Evasion

In a host environment compromised by a Ring-0 adversary, standard execution paradigms are catastrophically vulnerable. Modern Endpoint Detection and Response (EDR) systems, debuggers, and malicious hypervisors rely on deterministic execution artifacts—specifically the Import Address Table (IAT) and static string heuristics—to map an application's behavior. To ensure the survival of the Zero-Trust cryptographic state, Panoptes operates as a self-obfuscating native enclave, actively evading operating system telemetry.

10.1 The Fallacy of Static Execution and IAT Eradication

When a standard compiled binary requests an OS-level function (e.g., `VirtualAlloc`), the compiler resolves this via the Import Address Table (IAT). In a hostile Ring-0 environment, adversaries deploy API Hooking (e.g., inline detours or IAT patching) to intercept these calls, allowing them to passively harvest plaintext buffers and cryptographic seeds.

Panoptes neutralizes this vector through Complete IAT Eradication. The native engine is compiled as a position-independent, freestanding payload with zero external dependencies. The IAT is systematically stripped during the linking phase, rendering the binary statically opaque to disassemblers like IDA Pro or Ghidra. By severing ties with the OS loader, Panoptes denies adversaries the ability to preemptively hook its execution path.

10.2 Blind Resolution: PEB Walking and the Ldr List

Without an IAT, Panoptes must locate essential system APIs dynamically at runtime. It achieves this by directly parsing the operating system’s internal data structures, bypassing user-mode telemetry entirely.

On Windows-based x86_64 architectures, the engine locates the base address of the `ntdll.dll` and `kernel32.dll` libraries by interrogating the Process Environment Block (PEB) [20]. The execution flow accesses the Thread Environment Block (TEB) via the `GS` segment register and extracts the PEB pointer. Panoptes walks the doubly-linked `InMemoryOrderModuleList` to find the base addresses of loaded modules, avoiding highly monitored functions like `GetModuleHandle`.

10.3 PEB Cloaking and Module Unlinking

To prevent memory scrapers from easily identifying the native enclave, Panoptes proactively unlinks its own module from the Process Environment Block (PEB). By parsing the `PEB_LDR_DATA` structure and modifying the forward and backward links of the `InLoadOrderModuleList`, `InMemoryOrderModuleList`, and `InInitializationOrderModuleList`, the DLL becomes invisible to standard OS-level diagnostic tools.

10.4 Double-Blind FNV-1a API Hashing

Once the base address of a target module is located, Panoptes must parse its Export Directory to find the specific function address. To prevent string-matching heuristics from flagging the engine’s intent, function names are never stored as plaintext strings in the `.rdata` section.

Instead, Panoptes utilizes a dynamically seeded 64-bit FNV-1a non-cryptographic hash function [22]. A build-specific salt is mathematically generated via bitwise rotation of the Chaos Seed:

$$build_salt = (chaos_seed \ll 7) \mid (chaos_seed \gg 57) \quad (19)$$

To enable Double-Blind Lookup and bypass OS-specific casing inconsistencies in exported symbols, the engine strictly forces all characters to lowercase via bitwise addition (`char + 32`) before absorbing them into the hash:

$$h = (h \oplus \text{tolower}(byte)) \times 0x100000001B3 \pmod{2^{64}} \quad (20)$$

If the computed hash matches a hardcoded, pre-computed constant corresponding to the desired API, the engine extracts the function pointer.

10.5 Dynamic Payload Assembly via MBA Generation

If symmetric keys or magic numbers (such as the ChaCha20 constants) are stored contiguously in memory, memory-carving tools can easily locate them via entropy analysis.

To solve this, Panoptes employs Build-Time MBA Metaprogramming. A custom Python pre-processor ingests the C source code before compilation. For every sensitive constant K_{target} , the script generates a highly complex Mixed Boolean-Arithmetic (MBA) expression [23]. For example:

$$K_{target} = (X + Y) - 2 \cdot (X \& Y) \quad (21)$$

These expressions are embedded directly into the `.text` section. The cryptographic keys do not exist at rest; they are dynamically assembled in the CPU registers during the exact microsecond they are required, and are immediately destroyed by the `secure_wipe` protocol. The registers are explicitly zeroed out (`_rA = 0; _rB = 0;`) post-assembly to prevent leakage to the stack.

10.6 Cross-Platform Evasion via Direct Syscalls

To evade user-mode API hooking on POSIX systems (Linux and macOS), Panoptes completely decouples from the standard C library (`libc`). All critical memory operations (e.g., `mmap`, `mprotect`) and file descriptors are invoked via direct, inline-assembly system calls.

On x86_64 Linux, the engine utilizes the `syscall` instruction. On Apple Silicon (ARM64), it issues direct kernel interrupts via the `svc 0x80` instruction. This architectural isolation neutralizes LD_PRELOAD library injections, ensuring the execution flow remains completely opaque.

10.7 JIT String Encryption (Rolling Avalanche)

Static strings provide trivial heuristics for reverse engineering. Panoptes eradicates plaintext metadata by encrypting all strings, file paths, and target module names using a Rolling Avalanche cipher at compile-time, derived from a CSPRNG. The decryption logic is embedded inline and relies on strict `& 0xFF` byte-casting to prevent Integer Promotion bugs.

10.8 Polymorphic Engine and Control Flow Flattening (CFF)

To neutralize persistent adversaries who share reverse-engineering signatures, Panoptes is inherently polymorphic. During the pre-compilation phase, a dedicated Python engine injects between 100 and 250 interconnected decoy functions into the source code. These simulate Fake Cryptography (e.g., RC4 KSA) and employ Control Flow Flattening (CFF) to flatten the execution graph, rendering static analysis of the call stack mathematically intractable.

10.9 Stealth Libc and Standard Memory Hook Evasion

Advanced EDR agents routinely detour functions like `memcpy` to sniff plaintext cryptographic keys. Panoptes eradicates this attack surface by completely detaching from the globally exposed `libc` symbols. Through explicit compiler macros, all memory management is redirected to localized, inline implementations executing byte-by-byte transformations strictly within the unhooked boundaries of the native enclave.

11 Decoy Memory and The Vault Topology

Even with Dynamic API evasion and an airgapped execution enclave, Panoptes must eventually materialize the symmetric decryption keys and plaintext game state in physical RAM. Against a Ring-0 adversary utilizing PCIe-based Direct Memory Access (DMA), standard memory encryption is insufficient. Panoptes implements a spatial obfuscation architecture known as The Vault.

11.1 The DMA Entropy-Scanning Threat

Modern forensic tools and DMA hardware do not blindly dump the entire physical RAM. Instead, they utilize entropy scanners to locate cryptographic material. Cryptographic keys stand out as high-entropy islands (≈ 8.0 bits/byte) within the memory landscape. Panoptes operates under a Needle-in-a-Needlestack paradigm to neutralize this heuristic.

11.2 High-Entropy Decoy Multiplexing

During initialization, Panoptes allocates an array $\mathbb{V}$ consisting of N identical Vault structures. Only one structure, $\mathbb{V}_{real}$, contains the actual cryptographic state. The remaining $N - 1$ structures are Decoy Vaults, continuously populated with pseudo-random high-entropy garbage generated by a non-blocking ChaCha20 keystream:

$$\forall i \in \{0, \dots, N - 1\}, H_{shannon}(\mathbb{V}_i) \approx 8.0 \text{ bits/byte} \quad (22)$$

This forces signatureless memory scanners into a combinatorial explosion, unable to isolate the valid state matrix asynchronously.

The exact memory layout of the **PanoptesVault** structure enforces strict domain separation:

Table 2: PanoptesVault Memory Layout

Structure Member	Size / Type	Architectural Function
<code>magic / poison</code>	$2 \times \text{unsigned int}$	Vault integrity anchors and branchless MBA failure states.
<code>last_tick_[0,1,2]</code>	$3 \times \text{unsigned long long}$	Timestamps for the Multithreaded Deadman Switch cross-vigilance.
<code>SESSION_[PRIV PUB]_KEY</code>	2×32 Bytes	Ephemeral X25519 KEM pairs (Zeroized upon Vault Lobotomy).
<code>HAND_ID</code>	16 Bytes	Unique cryptographic identifier for the current micro-ledger epoch.
<code>SHUFFLE_KEY_SHARE</code>	32 Bytes	XOR shard of the <code>MASTER_SHUFFLE_KEY</code> for deck decryption.
<code>POCKET_CARDS</code>	2 Bytes	Validated player hole cards.
<code>CURRENT_STREET</code>	4 Bytes	Integer pointer defining the current active betting round.
<code>STREET_TOKENS</code>	3×16 Bytes	Hash-ratcheted shards for Flop, Turn, and River decryption.
<code>ESCROW_CIPHERTEXT</code>	5 Bytes	ChaCha20 encrypted community cards pending token release.
<code>HAND_STATE_BLOCKCHAIN</code>	32 Bytes	512-bit Sponge accumulated state of all serialized actions.
<code>P2P_PUB_KEYS</code>	10×32 Bytes	Long-term public identity keys for active peers in the mesh.
<code>P2P_MAC_KEYS</code>	$10 \times 4 \times 64$ -bit	Pre-derived Poly1305 MAC keys for the N-MAC Swarm.
<code>P2P_COMMITMENTS</code>	10×32 Bytes	Phase 1 Hash-commitments of all active peers' seeds.
<code>capture_in_progress</code>	4 Bytes	Mutex flag for the visual anti-ESP telemetry routines.
<code>CACHED_JAR_STATE</code>	4×64 -bit	Asynchronous SipHash-2-4 accumulator for Binary Attestation.
<code>CACHED_LIB_HASH</code>	16 Bytes	Asynchronous SipHash-2-4 accumulator for Native Lib Attestation.

11.3 Hardware-Level Page Guarding and TOCTOU Mitigation

Even with decoy multiplexing, leaving the Vault continuously accessible in RAM presents a window of vulnerability to PCILeech-style DMA attacks. Panoptes mitigates this by aggressively toggling the hardware-level page protections.

Upon allocation, the Vault’s memory pages are explicitly marked as inaccessible using `mprotect(PROT_NONE)` on POSIX or `VirtualProtect(PAGE_NOACCESS)` on Windows. To prevent Time-of-Check to Time-of-Use (TOCTOU) DMA attacks during the brief window when memory must be read, the engine does not decrypt the state *in situ*.

Instead, Panoptes executes a highly optimized sequence:

1. The Vault’s page protection is temporarily elevated to `PROT_READ`.
2. The encrypted chunk is immediately copied from the physical Vault into the thread’s local execution stack via a stealth memory copy.
3. The Vault’s page protection is instantly reverted to `PROT_NONE`, mathematically minimizing the exposure window to a few CPU cycles.
4. The ChaCha20 decryption is performed offline on the local stack copy.

When writing to the state, the process is inverted: the plaintext is encrypted offline in the stack, the Vault is briefly opened for `PROT_WRITE`, the ciphertext is committed, and the Vault is immediately shut. This is followed by an immediate `secure_wipe` of the local keys and plaintext stack buffers. This architectural pattern renders the memory physically unreadable to the OS and DMA hardware for 99.9% of the application’s lifecycle, successfully defeating asynchronous carving attacks.

11.4 Compiler-Defeating Volatile Memory Barriers

To guarantee the physical annihilation of plaintext keys and defeat Dead-Code Elimination (DCE), Panoptes implements a custom `secure_wipe` routine. This routine casts the target memory to a volatile pointer and executes a full compiler memory barrier:

```
__asm__ __volatile__("":: "r" (v): "memory");
```

This barrier instructs the compiler that memory has been arbitrarily modified, preventing instruction reordering and forcing the CPU to flush the zeroized bytes to physical RAM.

11.5 Branchless Target Resolution

Selecting $\mathbb{V}_{real}$ from the multiplexed array $\mathbb{V}$ utilizes MBA to derive the execution pointer branchlessly. Let T be the active index of the real vault, and X_i the current iteration index. The engine calculates a resolution mask μ_i :

$$\mu_i = ((X_i \oplus T) - 1) \gg 31 \quad (23)$$

The state pointer is then updated via bitwise accumulation across the entire array, forcing the CPU to access every single vault (real and decoy) in constant time:

$$P_{state} = \bigoplus_{i=0}^{N-1} (\&\mathbb{V}_i \text{ AND } \mu_i) \quad (24)$$

12 Hostile Attestation and The Spinlock Sentinel

The final layer of the Panoptes defense-in-depth architecture assumes that an adversary has successfully bypassed the JNI boundary and located the Vault execution context. Panoptes counters active attacks through Asynchronous Binary Attestation and Atomic Spinlock concurrency control.

12.1 Asynchronous Binary Attestation (SipHash-2-4)

To guarantee the integrity of the executing logic, the engine implements an internal, self-referential attestation mechanism utilizing the SipHash-2-4 pseudo-random function [10]. Dedicated asynchronous worker threads perform continuous disk I/O operations, reading the binaries in 1MB chunks and generating a cryptographic hash of the physical files (`.jar` and `.so/.dll/.dylib`).

The resulting SipHash is asynchronously committed to the encrypted Vault. If an adversary attempts to patch the binary on disk or hook an internal function, the hash diverges from the expected baseline. If a divergence is detected, the engine sets an internal `hash_worker_error` flag. This error branchlessly corrupts the outgoing KEM and P2P MACs in subsequent payloads, effectively ostracizing the compromised node from the mesh without triggering a local crash that could aid reverse engineering.

12.2 Concurrency Attacks and The Spinlock Sentinel

A sophisticated Ring-0 adversary will attempt to exploit the multi-threaded nature of the JVM by pausing the main thread at the exact moment a cryptographic key is generated and immediately spawning a rogue thread to read the plaintext Vault (a TOCTOU attack).

Standard OS-level mutexes (e.g., `pthread_mutex_lock`) are insufficient because they require context switches to the kernel, which a Ring-0 adversary controls. Panoptes resolves this by implementing a custom Spinlock Sentinel utilizing atomic, hardware-level CPU instructions [19]. The engine guards access to the enclave using the `--sync_lock_test_and_set` intrinsic, forcing other logical cores into a highly optimized `PANOPTES_SPINLOCK_BUSY_CYCLES` busy-wait loop. If the spinlock fails to acquire the lock rapidly, it degrades to a `usleep` fallback to prevent hard deadlocks within the JVM execution threads.

12.3 Multithreaded Cross-Vigilance (Deadman Switch)

To mitigate thread suspension attacks via a JDWP debugger or hardware breakpoints (DR0-DR3), Panoptes spawns up to three distinct guardian threads. These guardians engage in continuous cross-vigilance, recording their active timestamps into the encrypted Vault.

During execution, each thread validates the progression of the local system clock against the hardware's internal cycle counter via the `RDTSC` instruction (or `cntvct_el0` on ARM64). If an attacker suspends the primary thread, the `RDTSC` counter will exhibit an anomalous drift. If this drift exceeds the `PANOPTES_MAX_RDTSC_DRIFT` threshold, or if a guardian fails to emit a heartbeat within the `DEADMAN_THRESHOLD_MS` window, the surviving threads mathematically poison the Vault.

Furthermore, relying solely on temporal drift is insufficient against stealth hypervisors. The guardian threads proactively query the execution context of the primary thread at the OS level (e.g., via `GetThreadContext` on Windows). They directly evaluate the hardware debug registers (DR0 through DR3). If any of these hardware breakpoints are set, an MBA mask is immediately evaluated to irrevocably poison the cryptographic state, completely neutralizing silent execution halting.

12.4 Environmental Telemetry and Anomalous Memory Scanner

To prevent telemetry spoofing via containerized cloning, the OS Radar extracts a deterministic Hardware Fingerprint (HWID). It computes a 64-bit FNV-1a hash over the Volume Serial and Computer Name (Windows), `/etc/machine-id` (Linux), or `kern.uuid` (macOS), cryptographically anchoring the radar payload to the physical silicon.

Panoptes features a comprehensive OS Radar that conducts continuous telemetry of the host system's security posture. The radar dynamically assembles an `NtQueryVirtualMemory` gadget to scan the process memory space. It actively hunts for anomalous pages flagged with `PAGE_EXECUTE_READWRITE` (0x40) or `PAGE_EXECUTE_READ` (0x20) that are strictly `MEM_PRIVATE` (0x20000)—a definitive signature of unbacked shellcode injection or EDR inline hooks.

Furthermore, the radar scans the process memory space for inline hooks (e.g., 0xE9 or 0xEB JMP instructions overriding `CreateFileA` or `open`) and unbacked `rwxp` memory segments indicating shellcode. Additionally, the radar autonomously detects virtualized environments via CPUID leaf 0x40000000. At the OS level, it interrogates kernel security policies:

- **Windows:** Queries `NtQuerySystemInformation` to verify Hypervisor-Enforced Code Integrity (HVCI) status.
- **macOS:** Probes `csr_get_active_config` to determine if System Integrity Protection (SIP) is disabled.
- **Linux:** Verifies Kernel Lockdown enforcement by parsing `/sys/kernel/security/lockdown`. Furthermore, it actively scans `/proc/self/maps` for anomalous memory pages marked as `rwxp` or `r-xp` that lack a backing file path, definitively identifying unbacked shellcode injections.

12.5 OS-Specific Anti-Tracing and Introspection Defenses

Beyond static memory scanning, Panoptes actively neutralizes process-level debuggers and dynamic instrumentation frameworks by leveraging OS-specific telemetry:

- **POSIX (Linux):** The engine actively scans for `LD_PRELOAD` environment variables to prevent library hooking. It invokes `prctl(PR_SET_DUMPABLE, 0)` to block Ring-0 core dumps, and passively parses `/proc/self/status` to detect non-zero `TracerPid` values without raising system-call alarms.
- **POSIX (macOS):** It explicitly repels debuggers via `ptrace(PT_DENY_ATTACH, 0, 0, 0)`. Additionally, it evaluates the `CS_DEBUGGED` flag via `csops` and sweeps `task_get_exception_ports` to catch malicious Mach exception handlers.
- **Windows:** The engine dynamically routes `CheckRemoteDebuggerPresent` and evaluates the `DebugPort` via `NtQueryInformationProcess`, while scanning for known JVM instrumentation modules like `jdwp.dll` and `instrument.dll`.

12.6 Visual Telemetry and Anti-ESP Mechanisms

To proactively defend against Ring-3 visual overlays (Extra Sensory Perception or ESP cheats) that intercept the rendering pipeline, the engine implements a dedicated routine utilizing native graphical APIs (e.g., GDI+ on Windows). This routine locates the target game window, captures the pixel buffer, and converts the color space to grayscale (luminance) using fast bitwise arithmetic $(r * 77 + g * 150 + b * 29) \gg 8$ to minimize bandwidth. The resulting snapshot is compressed into a JPG and returned to the JVM for transmission over the encrypted KEM channel, providing the host node with cryptographic proof of the client's visual state.

Crucially, prior to capturing the frame on Windows architectures, the engine invokes `SetWindowDisplayAffinity` with the `WDA_EXCLUDEFROMCAPTURE` (0x11) flag. This forces the OS Desktop Window Manager (DWM) to render the game window as a black box to all external screen-recording or broadcasting software (e.g., OBS, Discord). This preemptively neutralizes external OCR overlay cheats and stream-sniping, while allowing the native enclave to securely capture its own internal HBITMAP.

12.7 Local Network Attestation (Anti-Proxy / MITM)

A sophisticated Ring-0 attacker might attempt to bypass the decentralized P2P cryptography by deploying a localized proxy loopback (e.g., Wireshark or a custom MITM proxy) to intercept and manipulate Megapackets before they are serialized.

To neutralize this, Panoptes implements a Layer 1 Network Attestation protocol. The native C enclave actively interrogates the host’s internal network state to verify socket integrity. Depending on the architecture, the engine parses `/proc/net/tcp` and `/proc/net/tcp6` (Linux), iterates `proc_pidinfo` with `PROC_PIDLISTFDS` (macOS), or retrieves the `GetExtendedTcpTable` (Windows).

The engine verifies the specific process ID (PID) owning the socket and validates that no anomalous local proxy holds a connection state matching the designated P2P port. If an unverified loopback connection is detected, the environmental attestation fails, and the local Vault refuses to decipher incoming Megapackets.

13 Evaluation: The “Total Siege” Framework

Evaluating a Zero-Trust engine designed for a Ring-0 adversary requires abandoning standard application-layer penetration testing. Instead, we evaluate Panoptes under a theoretical and architectural framework we define as the **Total Siege**. In this model, the evaluator assumes the role of a hypervisor-level attacker equipped with PCIe DMA hardware, JDWP runtime debuggers, and CPU performance counter monitors.

The engine is systematically evaluated against a rigorous 40-point endurance and security matrix embedded natively into the Java testing suite. Table 3 details the complete audit manifest:

Table 3: Total Siege 40-Point Audit Matrix

Audit ID	Attack Vector / Simulation	Engine Response & Status
AUDIT 00	Tactical Path Parsing Stress Test	[PASS] Natively resolved Host JAR location despite obfuscated spacing/quotes, mitigating path-injection.
AUDIT 01	Physical JAR Attestation	[PASS] Runtime context and JAR signature binding validated.
AUDIT 02	PFS Session Key Exchange	[PASS] Ephemeral Key Encapsulation Mechanism strictly isolated in C.
AUDIT 03	Entropy Vault Persistence	[PASS] Entropy state safely exported and restored from disk.
AUDIT 04a	Network Attestation (L1)	[PASS] JNI crypto bridge operational via IPv4 / mapped IPv6 sockets.
AUDIT 05a	P2P Zero-Trust Mesh (L2 - Human)	[PASS] Mesh KEM challenge payload successfully decrypted.

Continued on next page

Table 3 – continued from previous page

Audit ID	Attack Vector / Simulation	Engine Response & Status
AUDIT 05b	P2P Zero-Trust Mesh (L2 - Bot)	[PASS] Bot verified via server proxy attestation.
AUDIT 06	Tactical Screenshots & Anti-flood	[PASS] Valid native GDI+ visual capture generated; antiflood cooldown functional.
AUDIT 07	Hostile JVM Intrusion Test	[PASS] JDWP attachment immediately triggered <code>check_tracer()</code> , detonating Vault.
AUDIT 08	System Radar Telemetry	[PASS] Telemetry (HVCI, SIP, TracerPid) successfully extracted and decrypted.
AUDIT 09	Megapacket Gen. & Commitments	[PASS] Host correctly derived deck permutation constraints.
AUDIT 10	Blind Ingestion	[PASS] Megapacket successfully ingested without deserialization exploits.
AUDIT 11	Bot KEM Extraction	[PASS] Bot cards cleanly recovered; strictly passed boundary checks.
AUDIT 12	Shuffle Determinism	[PASS] Fisher-Yates keystream permutation 100% deterministic across platforms.
AUDIT 13	Consensus Evolution	[PASS] Street evolved; branchless token extraction successfully decrypted Escrow.
AUDIT 14	Atomic Action Chain	[PASS] Action chain successfully anchored to the Sponge micro-ledger.
AUDIT 15	Forensic Verification	[PASS] Master seed completely and natively reconstructed from XOR-shards.
AUDIT 16	Vault Resurrection	[PASS] Event sourcing sync verified.
AUDIT 17	Session Isolation	[PASS] Cryptographic sessions actively survived hand transitions.
AUDIT 18	Multi-Hand State Reset	[PASS] Engine amnesia-safe. No residual poison carried to subsequent hands.
AUDIT 19	Concurrency Spinlock Stress	[PASS] 50 threads over 500 iterations survived without deadlocks via Spinlock Sentinel.
AUDIT 20	JNI Boundary Hardening	[PASS] Native engine safely deflected malformed arrays.
AUDIT 21	OOB Heap Fuzzing	[PASS] 100,000-byte buffer injection caught. Vault natively sealed (0 SIGSEGV).
AUDIT 22	P2P Replay Nonce	[PASS] Replay cache successfully caught duplicate signature injection.
AUDIT 23	Bot Signature Identity Mutation	[PASS] Bot authorized strictly via its specific signature on the Blockchain.
AUDIT 24	Identity Corruption (Host vs Bot)	[PASS] Host/Bot internal identities completely isolated within the Vault.
AUDIT 25	DOS Attestation Saturation	[PASS] KEM engine survived 512 iteration flood via $O(1)$ cooldown rejection.
AUDIT 26	Low-Order Point Injection	[PASS] Branchless math correctly identified and rejected weak X25519 scalars.

Continued on next page

Table 3 – continued from previous page

Audit ID	Attack Vector / Simulation	Engine Response & Status
AUDIT 27	Out-of-Phase Action	[PASS] Engine rejected desynchronized MAC payloads.
AUDIT 28	Double Ingestion	[PASS] Vault blocked malicious overwriting of active, sealed states.
AUDIT 29	Consensus Poisoning	[PASS] Branchless decryption with an invalid Street Key reliably poisoned the Vault.
AUDIT 30	Time-Travel Streets	[PASS] Dead vault correctly rejected chronologically impossible action requests.
AUDIT 31	Sponge Collusion	[PASS] Forensic engine successfully rejected an N-1 partial key collusion attempt.
AUDIT 32	Tactical State Retention	[PASS] Escrow arrays strictly preserved during continuous key share extraction.
AUDIT 33	Vault Lobotomy	[PASS] Vault fully sterilized; Ephemeral keys wiped with zero post-match traces.
AUDIT 34	Asymmetric Dealing	[PASS] Replay validation proved deck was mathematically pristine.
AUDIT 35	OS-Level DACL Kernel Lock-down	[PASS] <code>advapi32.dll</code> injected <code>DENY_ACCESS</code> ACE; blocked external <code>OpenProcess</code> .
AUDIT 36	Deadman Switch Cross-Vigilance	[PASS] Rigor mortis detected via <code>RDTSC</code> drift; primary thread invoked MBA poisoning.
AUDIT 37	L1 Time Mutation	[PASS] Attestation cleanly rejected timestamp manipulation via underflow math.
AUDIT 38	TOC/TOU File Lock Evasion	[PASS] OS actively denied write access to the JAR due to Native Engine file-locking.
AUDIT 39	RNG Statistical Uniformity	[PASS] Modulo bias completely eradicated over 5000 uniform shuffle iterations.
AUDIT 40	KEM Ephemeral Key Reuse	[PASS] KEM ciphertext guarantees Perfect Forward Secrecy per active session.
PHASE 3	Endurance Marathon	[PASS] Sustained 500 consecutive hands simulating JVM network stutter without panic.

13.1 Resistance to Asynchronous Memory Extraction (DMA)

Under a simulated PCILeech DMA attack, Panoptes demonstrates absolute architectural resistance through two mechanisms:

1. **Hardware-Level Page Guarding:** The Vault is mathematically inaccessible to DMA sweeps for 99.9% of its lifecycle, repelling asynchronous reads with page faults.
2. **Sub-Millisecond Zeroization:** By explicitly invoking volatile memory barriers, Panoptes guarantees that true keys exist in the stack strictly for the duration of the cryptographic operation ($\approx 150,000$ clock cycles). Asynchronous DMA sweeps are mathematically too slow to capture the ephemeral state before its physical annihilation via `secure_wipe`.

13.2 Resilience Against Execution Introspection

When subjected to JVM-level introspection via JVMTI agents or JDWP attachments [24], the Panoptes JNI Airgap remains uncompromised. Because the native engine explicitly rejects array pinning (`GetPrimitiveArrayCritical`) in favor of physical copies into stealth-allocated `PAGE_READWRITE` buffers, the JVM’s Garbage Collector and debugger APIs are completely blind to the cryptographic mutations. Furthermore, the Dynamic API Hashing and PEB cloaking successfully evade automated EDR hooking heuristics.

13.3 Micro-Architectural Side-Channel Mitigation

Panoptes inherently defeats timing and execution side-channels. The use of MBA for length validation ensures a uniform execution pipeline devoid of vulnerable branch instructions. Furthermore, manual pointer arithmetic ensures deterministic memory footprints, neutralizing heuristic-based cache monitoring algorithms by explicitly denying exploitable memory padding.

13.4 Performance Overhead Analysis

The implementation of extreme defense-in-depth mechanisms introduces unavoidable computational overhead. The generation of dynamic MBA expressions, the continuous calculation of SipHash-2-4 attestations, and the `CLFLUSH` serialization incur a significant penalty compared to standard OpenSSL/BoringSSL implementations.

However, in the context of the CoronaPoker [26] P2P network, this latency is strictly confined to the transition boundaries of game states (e.g., shuffling, dealing, and revealing streets). Because the X25519 KEM and ChaCha20-Poly1305 primitives are deeply optimized at the assembly level, the total state-transition latency remains in the sub-millisecond range (< 1 ms). This is vastly superior to the 12-to-15-second block latency of Web3 Smart Contracts [4], proving that Panoptes achieves Ring-0 security without sacrificing the real-time interactivity required for competitive gaming.

14 Conclusion

The digital card gaming industry has long been paralyzed by a false dichotomy: accept the catastrophic trust vulnerabilities of centralized servers, or accept the prohibitive latency and public transparency of decentralized blockchain ledgers.

In this paper, we introduced Panoptes, a highly optimized, Zero-Trust cryptographic engine that fundamentally shatters this paradigm. By replacing the centralized authority with a peer-to-peer X25519 Key Encapsulation Mechanism and a Sponge-based consensus micro-ledger, Panoptes guarantees a mathematically provable, fair game state.

Crucially, we have demonstrated that decentralized cryptography is only viable if it can survive deployment on hostile hardware. Through the implementation of a JNI Airgap, complete Import Address Table (IAT) eradication, hardware-level decoy multiplexing, and branchless MBA execution, Panoptes successfully defends its ephemeral cryptographic state against a persistent Ring-0 adversary.

Panoptes proves that absolute game integrity does not require a trusted third party, nor does it require a global, slow-moving blockchain. By treating the client’s own physical memory as a hostile battleground, we have established a new architectural standard for real-time, high-stakes decentralized applications.

Declarations

AI Disclosure: Generative AI has been utilized for linguistic refinement and structural organization. The author maintains full responsibility for the technical architecture and cryptographic protocol design.

References

- [1] A. Shamir, R. L. Rivest, and L. M. Adleman, “Mental Poker,” in *The Mathematical Gardner*, D. A. Klarner, Ed. New York: Springer, 1981, pp. 37–43.
- [2] J. Yan and B. Randell, “A Systematic Classification of Cheating in Online Games,” in *Proceedings of the 4th ACM SIGCOMM Workshop on Network and System Support for Games (NetGames '05)*, 2005, pp. 1–9.
- [3] A. Barnett and N. P. Smart, “Mental Poker Revisited,” in *Cryptography and Coding*, Springer, 2003, pp. 370–383.
- [4] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” *Ethereum Project Yellow Paper*, vol. 151, pp. 1–32, 2014.
- [5] A. Langley, M. Hamburg, and S. Turner, “Elliptic Curves for Security,” IETF, RFC 7748, Jan. 2016.
- [6] P. L. Montgomery, “Speeding the Pollard and Elliptic Curve Methods of Factorization,” *Mathematics of Computation*, vol. 48, no. 177, pp. 243–264, 1987.
- [7] D. J. Bernstein, “ChaCha, a variant of Salsa20,” in *SASC 2008: The State of the Art of Stream Ciphers*, 2008.
- [8] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” IETF, RFC 8439, Jun. 2018.
- [9] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Sponge Functions,” in *Ecrypt Hash Workshop 2007*, 2007.
- [10] J.-P. Aumasson and D. J. Bernstein, “SipHash: A Fast Short-Input PRF,” in *Progress in Cryptology INDOCRYPT 2012*, Springer, 2012, pp. 489–508.
- [11] D. E. Knuth, *The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms*. Boston, MA: Addison-Wesley Longman Publishing Co., Inc., 1997.
- [12] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.
- [13] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 1999, pp. 173–186.
- [14] T. Hund, C. Willems, and T. Holz, “Practical Timing Side Channel Attacks against Kernel Space ASLR,” in *2013 IEEE Symposium on Security and Privacy (S&P)*, 2013, pp. 191–205.
- [15] M. Lipp et al., “Meltdown: Reading Kernel Memory from User Space,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 973–990.

- [16] J. Van Bulck et al., “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution,” in *27th USENIX Security Symposium*, 2018, pp. 991–1008.
- [17] B. Gubser et al., “Dark Forest: A zkSNARK-based MMO,” 2022.
- [18] P. C. Kocher, “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems,” in *Advances in Cryptology (CRYPTO ’96)*, Springer, 1996, pp. 104–113.
- [19] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
- [20] M. Russinovich, D. Solomon, and A. Ionescu, *Windows Internals, Part 1 (6th Ed.)*. Microsoft Press, 2012.
- [21] U. Frisk, “PCILeech: Direct Memory Access (DMA) Attack Software,” DEF CON 24, Las Vegas, NV, 2016.
- [22] G. Fowler, L. C. Noll, and K. Vo, “Fowler-Noll-Vo Hash Function,” IETF Draft, 1991.
- [23] Y. Zhou, A. Main, J. X. Gu, and H. Johnson, “Information Hiding in Software with Mixed Boolean-Arithmetic Transforms,” in *Information Security Applications (WISA)*, vol. 4867, Springer, 2007, pp. 61–75.
- [24] S. Liang, *The Java Native Interface: Programmer’s Guide and Specification*. Reading, MA: Addison-Wesley, 1999.
- [25] L. Sassaman, M. L. Patterson, S. Bratus, and M. E. Locasto, “Security Applications of Formal Language Theory,” *IEEE Systems Journal*, vol. 5, no. 3, pp. 481–491, 2011.
- [26] A. L. Vivar, “CoronaPoker: The Texas hold ’em NL videogame,” GitHub repository, 2020. [Online]. Available: <https://github.com/tonikelope/coronapoker>